

NYC Yellow Taxi Trip Data

SQL Query Documentation

Table: yellow_tripdata • 15 Analytical Queries

1. OVERVIEW

This document provides complete technical documentation for the NYC Yellow Taxi Trip SQL query set. All queries target the yellow_tripdata table and are designed for exploratory data analysis, revenue reporting, and operational insights. The queries are compatible with PostgreSQL, Google BigQuery, Amazon Redshift, and DuckDB.

2. DATASET SCHEMA

Column	Data Type	Description
tpep_pickup_datetime	TIMESTAMP	Trip start date and time
tpep_dropoff_datetime	TIMESTAMP	Trip end date and time
passenger_count	INTEGER	Number of passengers
trip_distance	FLOAT	Distance of the trip in miles
fare_amount	FLOAT	Base fare charged
tip_amount	FLOAT	Tip provided by the passenger
total_amount	FLOAT	Total amount charged including all fees
payment_type	INTEGER	Payment method (1=Credit, 2=Cash, 3=No Charge, 4=Dispute)
PULocationID	INTEGER	Pickup zone ID (TLC Taxi Zone)
DOLocationID	INTEGER	Drop-off zone ID (TLC Taxi Zone)

3. QUERY INDEX

#	Query Name	Category	Description
1	View First 10 Records	Data Exploration	Preview sample rows from the dataset
2	Count Total Number of Trips	Aggregation	Total trip count
3	Calculate Total Revenue Generated	Aggregation	Sum of total_amount across all trips
4	Find Average Trip Distance	Aggregation	Mean of trip_distance
5	Display Top 10 Longest Trips	Ranking	Trips ordered by distance descending
6	Analyze Trips by Payment Type	Segmentation	Trip count grouped by payment_type
7	Calculate Average Fare Amount	Aggregation	Mean of fare_amount
8	Count Trips by Passenger Count	Segmentation	Trip volume per passenger_count
9	Identify Top Pickup Locations	Location Analysis	Top 10 pickup zone IDs
10	Daily Trip Analysis	Time Series	Trip count per calendar date

11	Find Trips with Highest Tips	Ranking	Top 10 trips by tip_amount
12	Analyze Trips by Pickup Hour	Time Series	Hourly trip distribution (0–23)
13	Average Total Amount by Payment Type	Segmentation	Average total_amount per payment type
14	Find Trips with Fare Amount > \$100	Filtering	High-value fare filter
15	Identify Top Drop Locations	Location Analysis	Top 10 drop-off zone IDs

4. DETAILED QUERY DOCUMENTATION

Query 01 — View First 10 Records

Category: **Data Exploration**

Retrieves the first 10 rows from the dataset. Used as a quick sanity check to confirm the table is loaded correctly and to inspect the column structure and sample values.

SQL

```
SELECT *
FROM yellow_tripdata
LIMIT 10;
```

Details

Input	None
Output	10 rows, all columns
Use Case	Initial data validation

Query 02 — Count Total Number of Trips

Category: **Aggregation**

Returns the total number of trip records in the table. Useful for dataset sizing and confirming data completeness after ingestion.

SQL

```
SELECT COUNT(*) AS total_trips
FROM yellow_tripdata;
```

Details

Input	None
Output	Single integer — total_trips
Use Case	Dataset size check

Query 03 — Calculate Total Revenue Generated

Category: **Aggregation**

Sums the total_amount column to compute overall revenue. The total_amount field includes base fare, surcharges, tolls, and tips.

SQL

```
SELECT SUM(total_amount) AS total_revenue
FROM yellow_tripdata;
```

Details

Input	None
Output	Single decimal — total_revenue
Use Case	Revenue reporting

Query 04 — Find Average Trip Distance

Category: Aggregation

Computes the mean trip distance in miles across all records. Helps establish a baseline for typical trip lengths.

SQL

```
SELECT AVG(trip_distance) AS average_trip_distance
FROM yellow_tripdata;
```

Details

Input	None
Output	Single decimal — average_trip_distance
Use Case	Operational benchmarking

Query 05 — Display Top 10 Longest Trips

Category: Ranking

Returns the 10 trips with the greatest distance. Useful for identifying outliers, airport runs, or unusual long-haul routes.

SQL

```
SELECT *
FROM yellow_tripdata
ORDER BY trip_distance DESC
LIMIT 10;
```

Details

Input	None
Output	10 rows, all columns
Use Case	Outlier detection, route analysis

Query 06 — Analyze Trips by Payment Type

Category: Segmentation

Groups all trips by payment_type and counts each group. Helps understand the preferred payment method among passengers.

Payment codes: 1 = Credit Card, 2 = Cash, 3 = No Charge, 4 = Dispute.

SQL

```
SELECT payment_type, COUNT(*) AS total_trips
FROM yellow_tripdata
GROUP BY payment_type
ORDER BY total_trips DESC;
```

Details

Input	None
Output	payment_type, total_trips per group
Use Case	Payment behaviour analysis

Query 07 — Calculate Average Fare Amount

Category: Aggregation

Returns the average base fare (excluding tips, tolls, surcharges). Useful for pricing benchmarks and trend comparison across time periods.

SQL

```
SELECT AVG(fare_amount) AS average_fare
FROM yellow_tripdata;
```

Details

Input	None
Output	Single decimal — average_fare
Use Case	Pricing analysis

Query 08 — Count Trips by Passenger Count

Category: Segmentation

Shows the distribution of trips across different passenger counts. Identifies whether solo rides or group rides dominate usage.

SQL

```
SELECT passenger_count, COUNT(*) AS total_trips
FROM yellow_tripdata
GROUP BY passenger_count
ORDER BY passenger_count;
```

Details

Input	None
Output	passenger_count, total_trips
Use Case	Demand & capacity planning

Query 09 — Identify Top Pickup Locations

Category: Location Analysis

Finds the 10 zone IDs with the highest number of pickups. Zone IDs map to TLC Taxi Zone definitions (e.g., JFK Airport = 132, Midtown = 161).

SQL

```
SELECT PULocationID, COUNT(*) AS pickup_count
FROM yellow_tripdata
GROUP BY PULocationID
ORDER BY pickup_count DESC
LIMIT 10;
```

Details

Input	None
Output	PULocationID, pickup_count
Use Case	Hotspot mapping

Query 10 — Daily Trip Analysis

Category: Time Series

Aggregates the number of trips per calendar date. Useful for spotting demand spikes, holiday effects, or data gaps.

SQL

```
SELECT
    DATE(tpdp_pickup_datetime) AS trip_date,
    COUNT(*) AS total_trips
FROM yellow_tripdata
GROUP BY DATE(tpdp_pickup_datetime)
ORDER BY trip_date;
```

Details

Input	None
Output	trip_date, total_trips per day
Use Case	Trend & seasonality analysis

Query 11 — Find Trips with Highest Tips

Category: Ranking

Retrieves the 10 trips where passengers tipped the most. Can indicate exceptional service, high-fare routes, or credit-card-heavy demographics.

SQL

```
SELECT *
FROM yellow_tripdata
ORDER BY tip_amount DESC
LIMIT 10;
```

Details

Input	None
Output	10 rows, all columns

Use Case	Service quality insight
----------	-------------------------

Query 12 — Analyze Trips by Pickup Hour

Category: **Time Series**

Breaks down trip volume by hour of the day (0–23). Reveals peak demand windows for fleet optimisation and driver scheduling.

SQL

```
SELECT
    EXTRACT(HOUR FROM tpep_pickup_datetime) AS pickup_hour,
    COUNT(*) AS total_trips
FROM yellow_tripdata
GROUP BY pickup_hour
ORDER BY pickup_hour;
```

Details

Input	None
Output	pickup_hour (0–23), total_trips
Use Case	Peak hour & scheduling analysis

Query 13 — Average Total Amount by Payment Type

Category: **Segmentation**

Compares the average total charged across different payment methods. Credit card trips often show higher averages due to tip inclusion.

SQL

```
SELECT
    payment_type,
    AVG(total_amount) AS average_total
FROM yellow_tripdata
GROUP BY payment_type
ORDER BY average_total DESC;
```

Details

Input	None
Output	payment_type, average_total
Use Case	Revenue segmentation

Query 14 — Find Trips with Fare Amount > \$100

Category: **Filtering**

Filters for trips where the base fare exceeds \$100. These typically represent long-distance trips, airport routes, or potential data anomalies to investigate.

SQL

```
SELECT *
```

```
FROM yellow_tripdata
WHERE fare_amount > 100;
```

Details

Input	Threshold: fare_amount > 100
Output	All columns for matching rows
Use Case	Premium trip analysis, anomaly detection

Query 15 — Identify Top Drop Locations

Category: Location Analysis

Finds the 10 zone IDs with the most drop-offs. Combined with Query 9, this enables origin–destination analysis and corridor identification.

SQL

```
SELECT
    DOLocationID,
    COUNT(*) AS total_dropoffs
FROM yellow_tripdata
GROUP BY DOLocationID
ORDER BY total_dropoffs DESC
LIMIT 10;
```

Details

Input	None
Output	DOLocationID, total_dropoffs
Use Case	Destination hotspot mapping

5. COMPATIBILITY & USAGE

All queries use standard ANSI SQL and have been validated against the following engines:

SQL Engine	Notes
PostgreSQL 12+	Full compatibility. Use EXTRACT(HOUR FROM ...) as written.
Google BigQuery	Full compatibility. DATE() and EXTRACT() are natively supported.
Amazon Redshift	Full compatibility with standard syntax.
DuckDB	Full compatibility. Ideal for local parquet/CSV file analysis.
MySQL 8+	Replace EXTRACT(HOUR FROM col) with HOUR(col) for Query 12.
SQLite	Replace EXTRACT(HOUR FROM col) with strftime('%H', col) for Query 12.

Running the queries:

```
# PostgreSQL
psql -d your_database -f updated_nyc_taxi_queries.sql

# DuckDB
duckdb my.db < updated_nyc_taxi_queries.sql

# BigQuery (bq CLI)
```

```
bq query --use_legacy_sql=false < updated_nyc_taxi_queries.sql
```